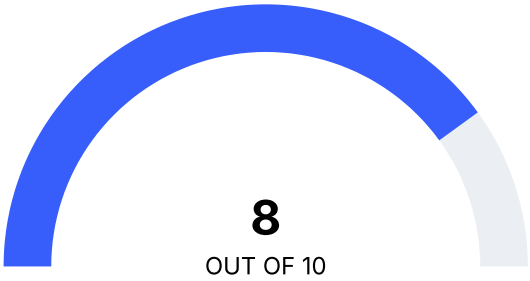
Candidate name	Job Title	Date
Amarnath Sm	Sr. Software Engineer - Fullstack	Friday, February 27th, 2026

Assessment

Interview score **8** OUT OF 10

AI Summary



Technical Skill	4 / 5
Problem Solving	3 / 5
Code Quality	3 / 5
Communication Rating	4 / 5

Interview Summary

Language/Framework... 4

Software Engineering 3

Frontend Concepts 3

HIGHLIGHTS

The candidate demonstrates strong backend fundamentals in C#/.NET along with working knowledge of React and frontend concepts. Shows practical understanding of concurrency concepts such as thread pool starvation, async blocking calls, `ConcurrentDictionary`, and `Interlocked`.
Has hands-on experience in API design, versioning, pagination, idempotency concepts, and cloud exposure (S3, Lambda, Terraform deployments). Some gaps remain in distributed system concepts (API Gateway, blue-green deployment, optimistic vs pessimistic locking), but overall performance aligns well with expectations for a strong IC full-stack engineer.
Suitable for Senior Software Engineer – Fullstack role.

Highlights
• Good explanation of thread pool starvation scenarios (.Wait/.Result causing blocking).
• Understands non-blocking async and importance of await.
• Strong grasp of `ConcurrentDictionary` and shared resource updates.
• Knows usage of `Interlocked` for atomic operations.
• Correct understanding of idempotency and API contracts(high level).
• Able to explain pagination design clearly.
• Good React fundamentals (re-render triggers, Virtual DOM, memoization).
• Exposure to AWS hosting, Terraform, Jenkins deployments.

SCOPE OF IMPROVEMENT

• Could not clearly differentiate race condition vs deadlock.
• Async deadlock explanation lacked depth (focused mostly on sync context).
• Blue-green deployment unfamiliar.
• API Gateway explanation missing.
• Locking strategies (optimistic vs pessimistic) unclear.
• Limited clarity on how rate limiting is enforced technically.
• Reconciliation explanation in React was partial.

Language/Framework Fundamentals

C# | ENTITY FRAMEWORK | MVC

4 OUT OF 5
Solid Performance, Exceeds Expectations

OVERALL FEEDBACK

- Strong C# fundamentals.
- Good understanding of thread pool starvation and blocking calls.
- Explained `ConcurrentDictionary` correctly for concurrent updates.

- Understood `Interlocked` usage for atomic increments.
- Knows async best practices (avoid blocking calls like .Wait/.Result).
- Slight confusion in race condition vs deadlock differentiation.
- MVC, WCF, ASP.NET experience .
- Testing experience present (though not deeply probed in this round).

Overall: Strong backend language fundamentals suitable for senior IC role.

Candidate has Good knowledge

QUESTION 1

Explain thread pool starvation -

ConcurrentDictionary -

Interlocked - i

How do you debug race conditions? - deadlock where you cannot recover ,

example race condition -
race condition vs deadlock

what was the situation -

What causes deadlocks in async code?

what is sync code -

FEEDBACK

Explain thread pool starvation - no more threads in thredpool , Scarios - using asynchronous calls , using blocking calls . .wait or . Result, high performing tasks and all blocking calls , non blocking calss and always await , static values , u can use interlocked , in async operations , dll , don't need to manage execution context

ConcurrentDictionary - lock management system , locks for every thread , resources are shared but can be updated concurrently , When should you use it - updating simultaneously , complex Value ?

Interlocked - increent static values , value is shared and multiple increase it

How do you debug race conditions? - deadlock where you cannot recover , 2 objects accessing at same time ,

example race condition - one using it another is trying to use

race condition vs deadlock - not sure

what was the situation -

What causes deadlocks in async code? he said deadlock occurs in sync

what is sync code -

Software Engineering

3 OUT OF 5
Performance Meets Expectations

OVERALL FEEDBACK

REST API design understanding solid.
- Correct explanation of PUT vs PATCH.
- Good understanding of idempotency concept.
- Pagination design explained clearly.
- Rate limiting concept known but enforcement mechanics unclear.
- Hosting knowledge via IIS and AWS (Terraform + Jenkins).
- AWS exposure (S3, Lambda) present but limited hands-on testing clarity.
- API Gateway concept not clearly articulated.
- Blue-green deployment unfamiliar.
- Locking strategy knowledge limited.

Overall: Strong application-level engineering, moderate distributed systems depth.

Candidate has Basic knowledge

QUESTION 1

REST API - What is the other type , RPC?
PUT vs PATCH difference?
What is idempotency?

How do you implement API versioning?

what are breaking changes ?

What is a API contract -

How do you design pagination?

How do you implement rate limiting?

how do you host your controllers -
how application is hosted in AWS

Cloud - implementing S3 , Lambda functions ----- How did it test it ?

what is an API Gateway -----

What is blue-green deployment?

Optimistic vs pessimistic locking?

FEEDBACK

REST API - What is the other type , RPC?
PUT vs PATCH difference? update is PUT , required resources ,
What is idempotency? the same req expects same response
POST needs action to make sure idempotency
How do you implement API versioning? - moving to new technology , we need versioning , we need to provide support ,

what are breaking changes ? if your reqeust and response can break client

What is a API contract - type OF Request and model , Request - Data contract ,

/v1/Controller /ACTioN Methods , POST : SaveScanDetailsAsync , Req = IN payload , you will have attributes , Res : Response Model , Headers : certain endpoints had ,

How do you design pagination? he could explain ,

How do you implement rate limiting? can set at controller level , how do you make sure , how this is maintianed , how controller level you know how many requests have come ,

how do you host your controllers - IIS , AWS - Dedicated ops team , how application is hosted in AWS - Terraforms , Jenkins deployed locally ,

Cloud - implementing S3 , Lambda functions ----- How did it test it ? he created local stack ? Difficult

what is an API Gateway -----

What is blue-green deployment? dont remember

Optimistic vs pessimistic locking?

Frontend Concepts

3 OUT OF 5
Performance Meets Expectations

OVERALL FEEDBACK

- React fundamentals solid (re-render triggers, props/state changes).
- Virtual DOM explanation correct.
- Memoization concept understood (prevent unnecessary re-renders).
- Forms handling basic explanation.
- Reconciliation explanation incomplete.
- TypeScript knowledge present but not deeply tested in this round.

Overall: Good functional frontend capability suitable for 30% frontend responsibility.

Candidate has Basic knowledge

QUESTION 1

What causes a React component to re-render?
What is virtual DOM?

What is reconciliation?

How do you handle forms in React?
What is memoization in React?

FEEDBACK

What causes a React component to re-render? - changes in component state , api result , parents prop changes

What is virtual DOM? it hold were done compare and updates , needed as lightweight , updates dom where it si needed ,

What is reconciliation?

How do you handle forms in React? - reat forms
What is memoization in React? - caching your data so that it doesnot rerender everytime ,

Interview Recording Link

Recording link : link